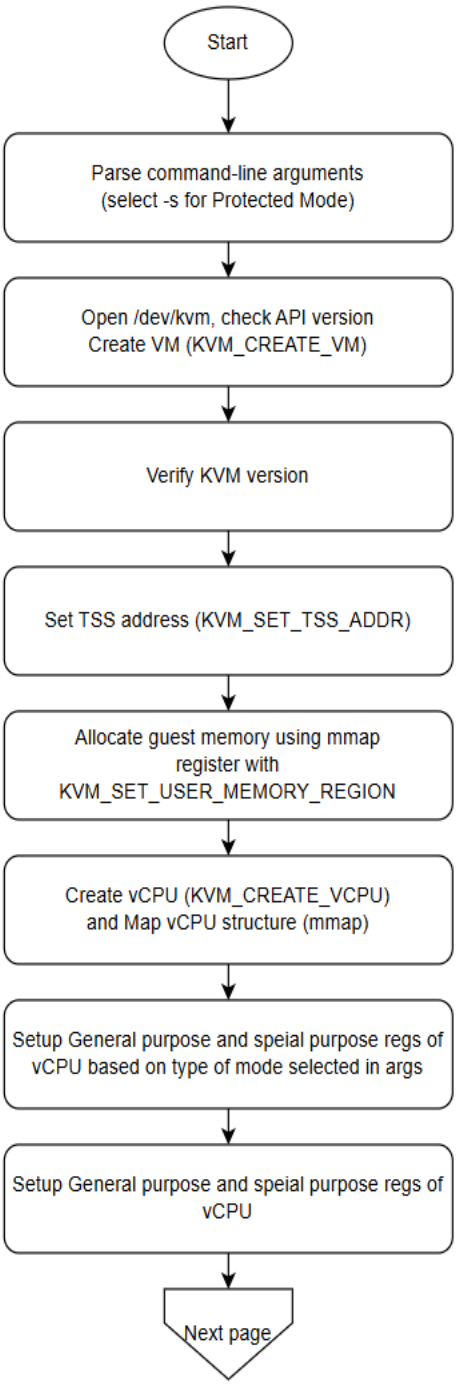
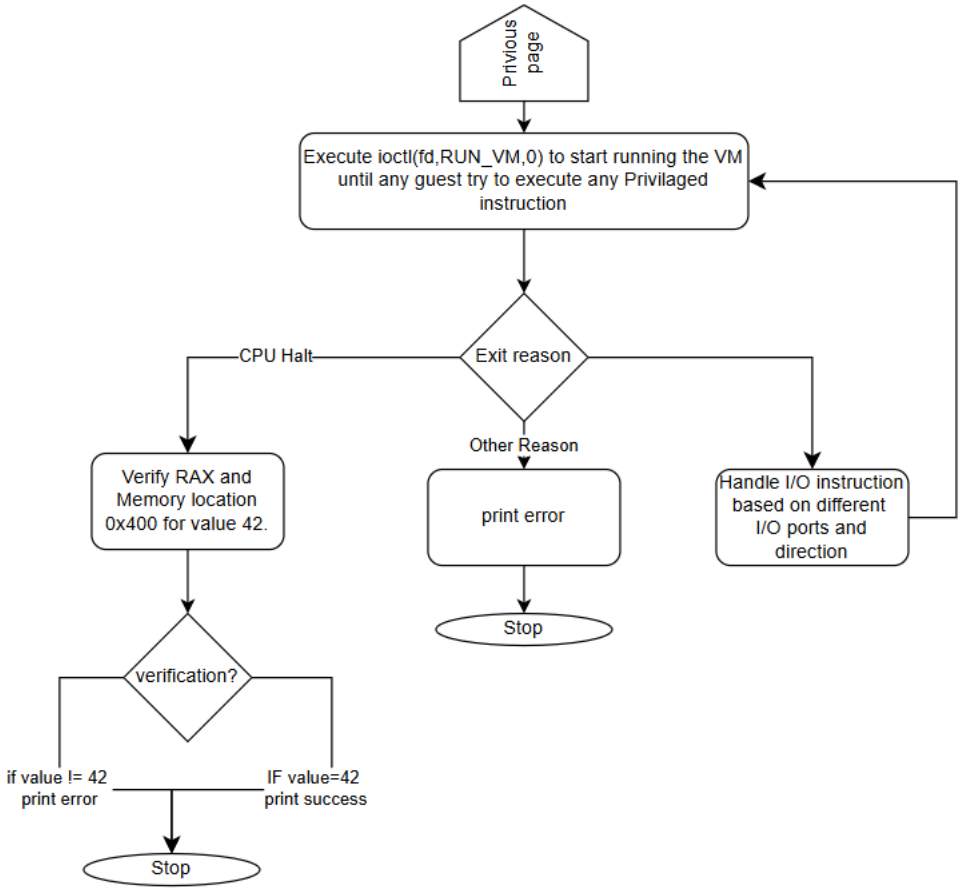
Assignment 2

Suyog Gatkal

Roll No. 24M0838

Flow chart:




Privious page
Execute ioctl(fd,RUN_VM,0) to start running the VM until any guest try to execute any Privilaged instruction
Exit reason
CPU Halt
Other Reason
Verify RAX and Memory location 0x400 for value 42.
print error
Handle I/O instruction based on different I/O ports and direction
Stop
verification?
if value != 42 print error
IF value=42 print success
Stop

1. 2 Explain the following code snippets provided in simple-kvm.c in detail.

2. a.

```
A extern const unsigned char guest64[], guest64_end[];
```

The keyword extern tells the compiler that these symbols are defined in another translation unit.

The compiler does not allocate storage for them here; it merely declares their existence so that they can be referenced later in the code.

These symbols act as markers for the start and end of the binary data. Linker script helps linker to mark start and end of each program (guest64, guest64_end etc) in the binary which can be used by programmer. These markers/pointers actually has address of start and end of the guest program where control has to jump to start running guest.

2. b

Essentially, this is part of process of setting up the initial page table and control registers of cpu of guest.

In long mode, the CPU uses a four-level paging system. For a minimal setup, we only populate the first entry at each level to cover a small region of memory.

```
pml4[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pdpt_addr;
pdpt[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pd_addr;
pd[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | PDE64_PS;
```

pml4[0], pdpt[0], pd[0] **:** These are the entries in the page table that points to base address of next level of page table

PDE64_PRESENT: Marks the entry as valid.

PDE64_RW: Allows read/write access.

PDE64_USER: Marks the entry as accessible from user mode.

PDE64_PS: This flag indicates that the entry maps a large page (typically 2MB) directly rather than pointing to a further page table.

Addr: Base address of next level page table.

```
sregs->cr3 = pml4_addr;
```

set up the CR3 register with page directory physical address.

```
sregs->cr4 = CR4_PAE;
```

This flag must be enabled to support the 64-bit paging mechanism. It allows the CPU to use 36-bit (or higher) physical addresses.

```
sregs->cr0 = CR0_PE | CR0_MP | CR0_ET | CR0_NE | CR0_WP | CR0_AM | CR0_PG;
```

CR0_PE: Enables protected mode.

CR0_MP, CR0_ET, CR0_NE, CR0_WP, CR0_AM: These flags set up additional CPU behaviors (like monitor coprocessor, numeric error handling, and write protection).

CR0_PG: Enables paging. With paging turned on, the CPU uses the tables set up earlier to translate virtual addresses.

```
sregs->efer = EFER_LME | EFER_LMA;
```

EFER_LME (Long Mode Enable): Tells the processor that it is allowed to operate in 64-bit mode.

EFER_LMA (Long Mode Active): Indicates that the processor has entered long mode.

2.c

```
vm->mem = mmap(NULL, mem_size, PROT_READ | PROT_WRITE, MAP_PRIVATE |
MAP_ANONYMOUS | MAP_NORESERVE, -1, 0);
```

Mmap is used here to allocate a block of memory for the virtual machine.

NULL:
By passing NULL as the first argument, the kernel chooses the starting address for the mapping.

mem_size:
This specifies the size (in bytes) of the memory region to be allocated. It represents the total amount of guest memory.

PROT_READ | PROT_WRITE:
These flags set the memory protection for the allocated region so that it can be both read from and written to.

MAP_PRIVATE:
Changes made to the mapped memory will not be visible to other processes and will not be written back to any file.

MAP_ANONYMOUS:
This flag indicates that the mapping is not backed by any file; instead, it allocates zero-initialized memory.

MAP_NORESERVE:
Tells the kernel not to reserve swap space for this mapping. This can be useful when the full memory might not be used immediately, though it comes with some risk if the memory is later overcommitted.

-1: Since the mapping is anonymous (not backed by a file), the file descriptor is set to -1.

0:
The offset in the file is set to zero, which is irrelevant for anonymous mappings.

```
madvise(vm->mem, mem_size, MADV_MERGEABLE);
```

This system call provides advice to the kernel about how to handle the memory region. It can influence decisions like paging, caching, or sharing.

vm->mem:
The starting address of the memory region previously allocated with mmap().

mem_size:
The size of the memory region.

MADV_MERGEABLE:
This flag advises the kernel that the pages in this memory region are eligible for Kernel SamePage Merging (KSM).

Kernel SamePage Merging (KSM):
KSM is a feature that scans memory for identical pages and merges them to save physical memory. This is particularly useful in virtualization environments where many virtual machines might be running similar or identical operating systems and applications. Merging these identical pages can reduce the overall memory footprint.

2.d

```
case KVM_EXIT_IO:
if (vcpu->kvm_run->io.direction == KVM_EXIT_IO_OUT && vcpu->kvm_run->io.port ==
0xE9)
{
      char *p = (char *)vcpu->kvm_run;
      fwrite(p + vcpu->kvm_run->io.data_offset,
      vcpu->kvm_run->io.size, 1, stdout);
      fflush(stdout);
      continue;
}
```

case KVM_EXIT_IO:

Part of switch case which decides the reason for exit into hypervisor. Here we are considering CPU is exited for IO.

vcpu->kvm_run->io.direction== KVM_EXIT_IO_OUT -:

gives direction of info on a port. (That is guest is writing to guest or reading from it.). Here we are dealing with guest trying to write on io port.

vcpu->kvm_run->io.port== 0xE9 :-

Gives the port IO port on which current operation is done.

In this If condition we are dealing with guest writing to 0xE9 port.

char *p = (char *)vcpu->kvm_run:

Obtaining a Pointer to the KVM_RUN Structure.

KVM _RUN Structure contains, among other things, an io substructure that holds details about the I/O operation, including:

data_offset: The offset within the kvm_run structure where the guest's output data is stored.

size: The number of bytes written by the guest.

fwrite(p + vcpu->kvm_run->io.data_offset, vcpu->kvm_run->io.size, 1, stdout);

p + vcpu->kvm_run->io.data_offset: extracts pointer to guest data. This is passed to fwrite to print data at that location.

vcpu->kvm_run->io.size: calculates the size of data passed by guest. This is passed to fwrite as size of individual element to be printed.

1: No of elements to be printed.

Stdout: direct fwrite to print to STDOUT stream.

fflush(stdout):

Immediately flushes the output buffer to ensure that the printed data is displayed promptly.

Continue: to skip this iteration of loop and move to next iteration of loop.

2.e

```
memcpy(&memval, &vm->mem[0x400], sz):
```

memcpy function used to copy data from one memory location to other.

Here we are copying the value present at OFFSET 0x400 in the guest address space into memval. Sz parameter denotes size of data to be copied. This copy operation is done to check guest behaviour. Guest is supposed to write value 42 at location 0x400 in his address space. Once guest is done with its operation hypervisor checks this value to understand the guest behaviour.

If the guest code fails to write the correct value, it may indicate issues such as:

- A bug in the guest code.
- Incorrect configuration or setup of the virtual CPU or memory.
- Problems with the hypervisor's handling of the guest's operations. This immediate check helps in diagnosing such problems early.